Universidad Panamericana

Artificial Intelligence for Video Games

Project 2

Shortest Path with Bidirectional Dijkstra

Team members

Rodrigo López Gómez	ID: 0262146
Valeria Pérez Maciel	ID: 0261983

Professor: Alfredo Emmanuel García Falcón

October 2026

Contents

1 Introduction **2**

1.1 Relation between `dijkstra.py` and `app.py` 2

2 Dijkstra's Algorithm **2**

2.1 Standard Dijkstra 2

2.2 Bidirectional Dijkstra (used in this project) 3

3 Data Structures, Classes, Methods and Attributes **5**

3.1 `dijkstra.py` — the algorithm 5

3.2 `app.py` — Flask backend and adapter 6

3.3 `static/app.js` — frontend 8

4 Instructions to Create a Graph **8**

5 Instructions to Run the Code **9**

6 Execution Results **10**

7 Conclusions **11**

1 Introduction

This project is a web application that lets the user build a directed, weighted graph visually and find the shortest path between a *start* node and a *destination* node using the **bidirectional Dijkstra** algorithm.

- **Algorithm:** `dijkstra.py`, written in Python using only the standard library (`heapq`, `itertools`).
- **Backend:** `app.py`, a Flask server.
- **Frontend:** `index.html`, `static/app.js` and `static/style.css`; the graph is drawn with Cytoscape.js.
- **Deployment:** Render, available at `https://dijkstra-python.onrender.com/`.

The identifiers of `dijkstra.py` are in Spanish: `Nodo` (node), `Arista` (edge), `Grafo` (graph), `peso` (weight), `inicio` (start) and `fin` (end). This report keeps the original names.

1.1 Relation between `dijkstra.py` and `app.py`

`dijkstra.py` is the **base of the algorithm** used by the application. It was written as a standalone module with its own classes (`Nodo`, `Arista`, `Grafo`) and can still be executed by itself.

The graphical interface does not work with those classes directly: the browser edits a lightweight JSON representation of the graph (nodes, roles, levels and connections). When the user presses **SOLVE**, `app.py` **rebuilds that graph as** `Grafo`/`Nodo` **objects** so it is compatible with the algorithm, calls `resolver()`, and translates the result (a list of `Nodo` objects and a cost) back into JSON so the interface can highlight the path. Therefore `app.py` acts as the adapter between the graphical part and the algorithm, while the shortest-path logic lives only in `dijkstra.py`.

`Browser (app.js)` $\xrightarrow{\text{JSON}}$ `app.py (Flask)` $\xrightarrow{\texttt{Grafo / Nodo}}$ `dijkstra.py` $\xrightarrow{\texttt{(path, cost)}}$ `app.py` $\xrightarrow{\text{JSON}}$ `Browser`

2 Dijkstra's Algorithm

2.1 Standard Dijkstra

Dijkstra's algorithm finds the shortest path from a source node s to every other node of a graph $G = (V, E)$ whose edge weights are **non-negative**, $w(u, v) \geq 0$.

The idea is greedy: the algorithm keeps a tentative distance $d[v]$ for every node and always processes the unvisited node with the smallest tentative distance. Because no edge has a negative weight, no later path can reach that node more cheaply, so its distance becomes final.

Algorithm 1 Standard Dijkstra

```
1: procedure DIJKSTRA(G, s)
2:     for all v ∈ V do
3:         d[v] ← ∞;    parent[v] ← null
4:     end for
5:     d[s] ← 0
6:     Q ← min-priority queue containing (0, s)
7:     while Q is not empty do
8:         (d_u, u) ← Q.pop_min()
9:         if u already visited then continue
10:        end if
11:        mark u as visited
12:        for all edge (u, v) with weight w do
13:            if d[u] + w < d[v] then                          ▷ relaxation
14:                d[v] ← d[u] + w;    parent[v] ← u
15:                Q.push(d[v], v)
16:            end if
17:        end for
18:    end while
19:    return d, parent
20: end procedure
```

The path to any node t is rebuilt by following *parent* from t back to s and reversing the list. Using a binary heap, the complexity is $O((|V| + |E|) \log |V|)$.

Its weakness is that the search grows like a "circle" around the source: to reach the destination it has to explore every node that is closer than the destination, in every direction.

2.2 Bidirectional Dijkstra (used in this project)

Bidirectional Dijkstra runs **two Dijkstra searches at the same time**:

Search	**Starts at**	**Follows**	**Structures**
Forward (f)	start node	outgoing edges (`aristas`)	`dist_f`, `padre_f`, `pq_f`, `vis_f`
Backward (b)	destination node	incoming edges (`entrantes`), reversed	`dist_b`, `padre_b`, `pq_b`, `vis_b`

Because the graph is directed, the backward search walks the edges "against the arrow": from a node it moves to the *origin* of each incoming edge. That is why every `Nodo` stores both its outgoing and incoming edges.

Meeting point. Every time one search relaxes an edge towards a node v, it also computes $d_{\text{this}}(v) + d_{\text{other}}(v)$. If the other search has already reached v, this is the cost of a complete path $s \rightsquigarrow v \rightsquigarrow t$. The best such value is kept in `mejor` and its node in `encuentro`.

Stopping condition. Let top_f and top_b be the smallest distances in each queue. Any path not

discovered yet must cost at least $top_f + top_b$, so the algorithm stops when

$$top_f + top_b \;\geq\; mejor.$$

Balancing. On each iteration the search whose queue has the smaller minimum is expanded, so both "circles" grow at a similar rate.

Algorithm 2 Bidirectional Dijkstra (`dijkstra_bidireccional`)

procedure BIDIRECTIONALDIJKSTRA(G, s, t)
  if $s = t$ **then return** $([s], 0)$
  end if
  $d_f[\cdot], d_b[\cdot] \leftarrow \infty$; $d_f[s] \leftarrow 0$; $d_b[t] \leftarrow 0$
  $Q_f \leftarrow \{(0, s)\}$; $Q_b \leftarrow \{(0, t)\}$
  $mejor \leftarrow \infty$; $encuentro \leftarrow$ null
  while Q_f and Q_b are not empty **do**
    if $top(Q_f) + top(Q_b) \geq mejor$ **then break**
    end if
    if $top(Q_f) \leq top(Q_b)$ **then**
      EXPAND(Q_f, d_f, d_b, outgoing edges)
    else
      EXPAND(Q_b, d_b, d_f, incoming edges)
    end if
  end while
  if $encuentro =$ null **then return** (null, ∞)
  end if
  $path \leftarrow$ reverse(follow $parent_f$ from $encuentro$)
  append (follow $parent_b$ from $parent_b[encuentro]$) to $path$
  return $(path, mejor)$
end procedure

procedure EXPAND(Q, d, d_{other}, edges)
  $(d_u, u) \leftarrow Q$.pop_min()
  if u already visited **then return**
  end if
  mark u as visited
  for all neighbor v of u through an edge of weight w **do**
    $new \leftarrow d_u + w$
    if $new < d[v]$ **then**
      $d[v] \leftarrow new$; $parent[v] \leftarrow u$; Q.push(new, v)
    end if
    if $new + d_{other}[v] < mejor$ **then**
      $mejor \leftarrow new + d_{other}[v]$; $encuentro \leftarrow v$
    end if
  end for
end procedure

The path is rebuilt in two halves: from the start to the meeting node following `padre_f` (and

reversing), and from the meeting node to the destination following `padre_b`, which already points towards the destination.

The worst-case complexity is the same as standard Dijkstra, $O((|V| + |E|) \log |V|)$, but in practice each search only covers about half of the distance, so two small "circles" explore far fewer nodes than one big circle. If there is no path, the queues run out without the searches meeting and the method returns `(None, inf)`.

3 Data Structures, Classes, Methods and Attributes

3.1 `dijkstra.py` — the algorithm

Only the elements that the application uses are described.

Class `Nodo` (node)

Attribute	Type	Description
`nombre`	`str`	Name of the node (unique in the app).
`aristas`	`list[Arista]`	Outgoing edges; used by the forward search.
`entrantes`	`list[Arista]`	Incoming edges; used by the backward search.

Method	Description
`__init__(nombre)`	Creates the node with empty edge lists.
`__repr__()`	Text representation, e.g. `Nodo(A)`.

Class `Arista` (directed, weighted edge)

The application never creates it directly; it is created inside `Grafo.agregar_arista`.

Attribute	Type	Description
`origen`	`Nodo`	Node where the edge starts.
`destino`	`Nodo`	Node where the edge ends.
`peso`	`int` / `float`	Weight (cost) of the edge, must be ≥ 0.

Method	Description
`__init__(origen, destino, peso)`	Creates the edge; raises `ValueError` if the weight is negative, because Dijkstra does not support negative weights.
`__repr__()`	Text representation, e.g. `A -> B (4)`.

Class `Grafo` (graph and algorithm)

Attribute	Type	Description
`nodos`	`list[Nodo]`	Every node in the graph.
`inicio`	`Nodo` or `None`	Start node.
`fin`	`Nodo` or `None`	Destination node.

Method	Description
`agregar_nodo(nodo)`	Adds the node if it is not already in the graph and returns it.
`agregar_arista(origen, destino, peso)`	Adds both nodes if needed, creates the `Arista`, appends it to `origen.aristas` and to `destino.entrantes`, and returns it.
`set_inicio(nodo)`, `set_fin(nodo)`	Set the start / destination node; raise `ValueError` if the node does not belong to the graph.
`resolver()`	Checks that start and destination are set and returns `dijkstra_bidireccional(inicio, fin)`, i.e. `(path, cost)`.
`dijkstra_bidireccional(origen, destino)`	Runs the bidirectional search (Section 2.2). Returns `(list[Nodo], cost)` or `(None, inf)`.
`_expandir(pq, dist, padre, visitados, dist_otro, adelante, mejor, encuentro, contador)`	One step of one search: pops the closest node, marks it visited, relaxes its edges (outgoing if `adelante` is `True`, incoming otherwise) and updates `mejor`/`encuentro` when it touches the other search. Returns the new `(mejor, encuentro)`.

Data structures inside `dijkstra_bidireccional`

Name	Structure	Purpose
`dist_f`, `dist_b`	`dict[Nodo, float]`	Best known distance from the start / to the destination.
`padre_f`, `padre_b`	`dict[Nodo, Nodo]`	Previous node in each search; used to rebuild the path.
`vis_f`, `vis_b`	`set[Nodo]`	Nodes whose distance is already final.
`pq_f`, `pq_b`	`list` used as a min-heap (`heapq`)	Priority queues with tuples `(distance, counter, node)`.
`contador`	`itertools.count()`	Tie-breaker: when two distances are equal the heap compares this number instead of the `Nodo` objects, which are not comparable.
`mejor`	`float`	Cost of the best complete path found so far.
`encuentro`	`Nodo`	Meeting node of that best path.

3.2 `app.py` — Flask backend and adapter

Graph state

Each browser session has its own graph, kept in memory as plain dictionaries that can be sent as JSON.

Structure	Description
`_GRAPH_STATES`	`dict[graph_id, state]`: all graphs, keyed by an id stored in the Flask session cookie.
`_STATE_LOCK`	`threading.RLock` that protects the states from concurrent requests.
`_MAX_NODES`	Limit of 40 nodes per graph.
state	`{"nodes": [...], "max_level": int, "solution": dict | None}`
node	`{"id", "name", "role", "level", "edges"}`. `role` is `"start"`, `"finish"` or `""`; `level` is the column where the node is drawn (start = 0, destination = last).
edge	`{"to": node_id, "weight": number}`, stored in the source node.
solution	`{"reachable", "pathIds", "pathNames", "cost", "message"}`.

Helper functions

Function	Description
`_new_node(name, role, level)`	Creates a node dictionary with a random id.
`_initial_state()`	New graph with `A` (start) and `B` (destination).
`_state()`	Returns the graph of the current session, creating it if needed.
`_state_response()`, `_error()`	Build the JSON responses (state, plus an error message when applicable).
`_find_node(state, id)`	Looks up a node by id.
`_invalidate_solution(state)`	Clears the previous result whenever the graph changes.
`_normalise_levels(state)`	Keeps start at level 0, regular nodes at level ≥ 1 and the destination one level after the last regular node.
`_parse_weight(value)`	Validates that a weight is a finite number ≥ 0.
`_next_name(state)`	Next free name: `A`…`Z`, then `N1`, `N2`, …
`_reassign_endpoint_if_needed()`	If the start or destination is deleted, promotes another node so both always exist.

HTTP endpoints

Route	Description
`GET /`	Serves `index.html`.
`GET /api/state`	Returns the current graph.
`POST /api/nodes`	Adds a node.
`PATCH /api/nodes/<id>`	Renames a node or changes its role.
`DELETE /api/nodes/<id>`	Deletes a node and every edge pointing to it.
`POST /api/nodes/<id>/level`	Moves a regular node one level left or right.
`POST /api/nodes/<id>/edges`	Adds (or updates) a connection `{to, weight}`.
`DELETE /api/nodes/<id>/edges/<to>`	Removes a connection.
`POST /api/solve`	Builds the `Grafo` and runs the algorithm.
`GET /health`	Health check used by Render.

Connection with the algorithm: `solve()`

This is where the interface and `dijkstra.py` meet. After validating that names are complete and unique and that there is exactly one start and one destination, `solve()` rebuilds the graph with the classes of `dijkstra.py`:

```
graph = Grafo()
node_objects = {node["id"]: Nodo(node["name"].strip()) for node in nodes}
for node in nodes:
    graph.agregar_nodo(node_objects[node["id"]])
for node in nodes:
    for edge in node["edges"]:
        destination = _find_node(state, edge["to"])
        graph.agregar_arista(node_objects[node["id"]],
                             node_objects[destination["id"]],
                             _parse_weight(edge["weight"]))
graph.set_inicio(node_objects[starts[0]["id"]])
graph.set_fin(node_objects[finishes[0]["id"]])
path, cost = graph.resolver()
```

The resulting list of `Nodo` objects is converted back to names and ids (`pathNames`, `pathIds`) and stored in `state["solution"]`.

3.3 `static/app.js` — frontend

- `state`: local copy of the graph returned by the server; every action sends a request and redraws using the new state.
- `api(path, method, body)`: sends requests to the backend and shows errors in a toast message.
- `card(node)`: builds each node card (name, role, connections and the form to add connections).
- `drawGraph()`: draws the graph with Cytoscape.js, placing each node in the column of its `level`. After solving, the path is drawn with thick lines and the rest of the graph is faded.
- `panels()`: shows the level-move panel of the selected node and the result card (total cost and path).

4 Instructions to Create a Graph

The page has three areas: the **node cards** (top), the **graph** drawing (bottom) and the **SOLVE** button (bottom right).

1. **Initial graph.** The page starts with node `A` (**Start**, green border) and node `B` (**Destination**, red border).
2. **Add nodes.** Click the + card at the end of the row. Each new node receives the next free name (`C`, `D`, ...). Up to 40 nodes are allowed.
3. **Rename a node.** Edit the text field of its card (at most 24 characters; names must be unique).
4. **Change the start or destination.** In the dropdown of a regular node choose **Start** or **Destination**; the previous one becomes a regular node.
5. **Add a connection (directed edge).** At the bottom of the *source* node's card select the target node, type the weight (a number ≥ 0) and press +. The connection appears as a chip "`Target · weight`". Adding the same connection again only updates its weight. For a two-way connection, add it from both cards.
6. **Remove a connection.** Click the × on its chip.
7. **Delete a node.** Click the × in the card header (only when the graph has more than two nodes). Its connections are deleted as well.
8. **Reorder the drawing (optional).** Click a regular node in the graph and use ← **Left** / **Right** → to move it to another column. This only changes the drawing, not the result.
9. **Solve.** Press **SOLVE**. The result card shows the **total cost** and the **path** (e.g. `A` → `C` → `D` → `B`) and the graph highlights it. If no path exists, it shows "No route found". Any later change to the graph clears the result.

5 Instructions to Run the Code

Running the project locally is not required: the application is deployed on Render and can be used directly at:

`https://dijkstra-python.onrender.com/`

Requirements

- Python 3.12 or newer.
- The packages listed in `requirements.txt` (Flask and Gunicorn).
- Internet connection in the browser (Cytoscape.js is loaded from a CDN).

Steps

```
# Create and activate a virtual environment
python3 -m venv venv
source venv/bin/activate        # Windows: venv\Scripts\activate

# Install the dependencies
pip install -r requirements.txt

# Run the application
python app.py
```

Then open `http://localhost:5000` in the browser. The address `http://localhost:5000/health` should return `{"status":"ok"}`.

Options

- **Different port:** `PORT=8000 python app.py`
- **Keep the session between restarts:** `SECRET_KEY=any-fixed-text python app.py`. Otherwise a new key is generated on each start and the browser gets a new, empty graph.
- **Run it as on Render (Gunicorn):** `gunicorn -w 1 -b 0.0.0.0:5000 app:app`. A single worker (`-w 1`) must be used because graphs are kept in memory; with several workers each one would hold a different copy.

Running only the algorithm

`dijkstra.py` can also be executed on its own and needs no dependencies:

```
python dijkstra.py
```

Its example builds a 20×20 grid with **400 nodes** (`N0` ... `N399`). Each node has a directed edge to the right and one downward, with weights between 1 and 9, which gives 760 edges. The program finds the shortest path from the top-left corner (`N0`) to the bottom-right corner (`N399`) and prints:

```
Nodos en el grafo: 400
Camino (39 nodos): N0 -> N1 -> N2 -> N3 -> N4 -> N24 -> N44 -> N45 ->
  N65 -> N85 -> N86 -> N106 -> N126 -> N127 -> N147 -> N167 -> N168 ->
  N169 -> N189 -> N190 -> N191 -> N192 -> N193 -> N213 -> N233 -> N234 ->
  N254 -> N274 -> N275 -> N295 -> N315 -> N316 -> N336 -> N356 -> N357 ->
  N377 -> N397 -> N398 -> N399
Costo total: 106
```

That is, the number of nodes in the graph, the path (39 nodes) and its total cost (106). The result was cross-checked with a standard Dijkstra over the same graph, which gives the same cost.

6 Execution Results

The following screenshots show the program running: the creation of the graph in the node cards, the drawing generated from them and the result of the algorithm after pressing **SOLVE**. The shortest path is drawn with thick lines and the nodes and edges that are not part of it are faded.

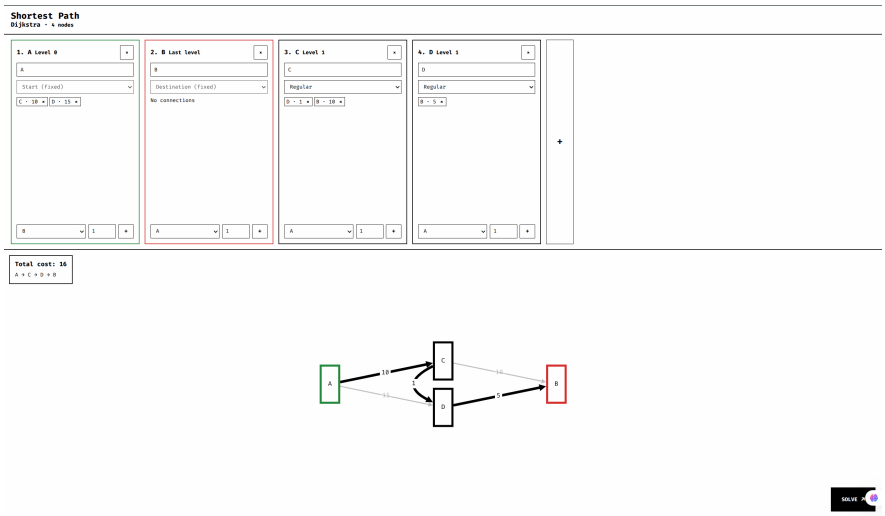


Figure 1: Graph with 4 nodes. From `A` the direct edges cost 10 (to `C`) and 15 (to `D`), and from `C` going to `B` directly costs 10. The algorithm finds that going through `D` is cheaper: `A` $\rightarrow$ `C` $\rightarrow$ `D` $\rightarrow$ `B` with a total cost of $10 + 1 + 5 = 16$ (the alternatives cost 20).

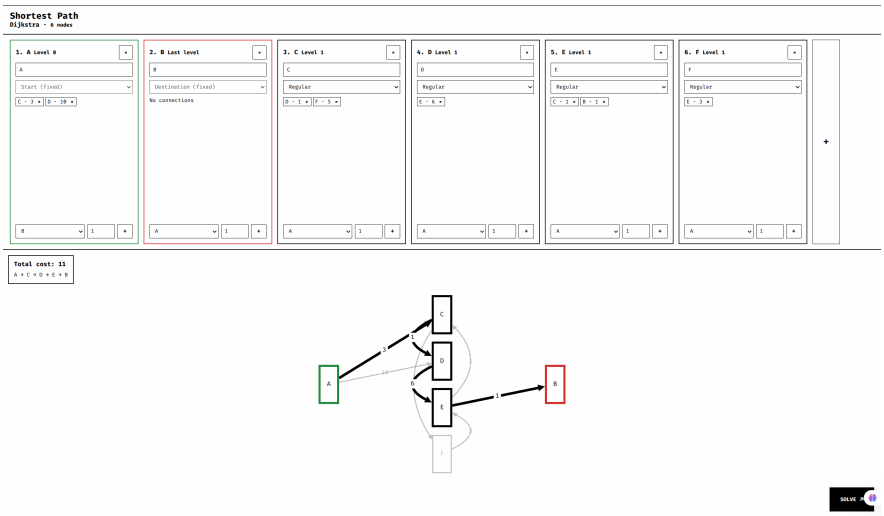


Figure 2: Graph with 6 nodes, including a node (`F`) that is not on the optimal route. Result: `A` → `C` → `D` → `E` → `B` with a total cost of $3 + 1 + 6 + 1 = 11$; the direct edge `A` → `D` (10) is not used.

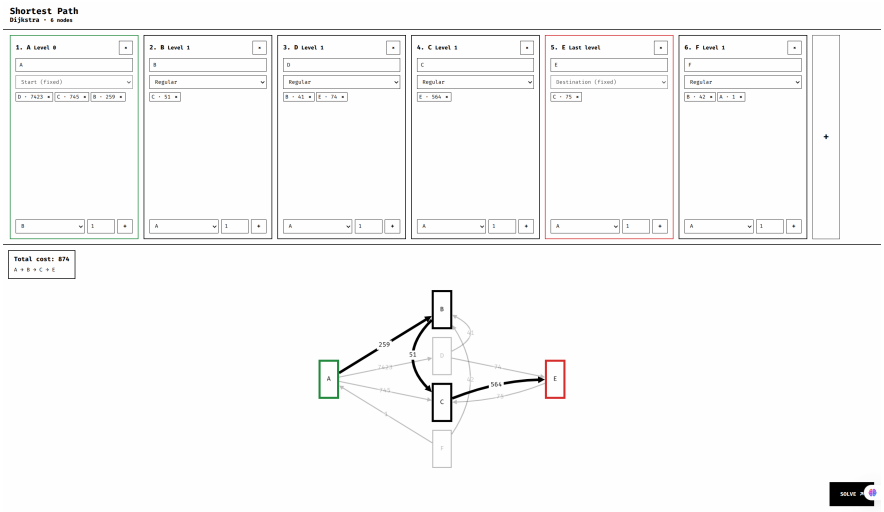


Figure 3: Graph with 6 nodes, large weights and edges that go back to the start (`F` → `A`) and from the destination (`E` → `C`). The destination was changed to `E`. Result: `A` → `B` → `C` → `E` with a total cost of $259 + 51 + 564 = 874$, cheaper than `A` → `C` → `E` (1309) or any route through `D` (7423 for the first edge alone).

7 Conclusions

Rodrigo López Gómez

For me, this project was a good opportunity to understand how an algorithm that we learn about in class can actually be used to create something functional. At first, the main goal was to implement Dijkstra's algorithm and make it find the shortest path between nodes, but as we worked on the project, I realized that there was more to it than just making the algorithm work. We also had

to think about how someone would use the application and how all the different parts would work together.

When we decided how to build the interface, we went with HTML and CSS, using a little bit of JavaScript for some interactions. We wanted to keep things simple and focus on making the application easy to understand and use without adding unnecessary complexity. For the backend, we used Python with Flask to connect the interface to the algorithm. Instead of making everything part of one large program, we kept the Dijkstra algorithm in its own file and used it like a small API. This way, the interface could send the graph information to the server, the algorithm could calculate the shortest path, and the result could be returned to the user. I think this was one of the most interesting parts of the project because it helped me understand better how the frontend and backend communicate in a real application.

We also deployed the application on Render, which was an important step because it allowed us to make the project available online instead of only running it on our own computers. This made the project feel more complete, since we were not just submitting code that worked locally, but an application that could actually be accessed and used. It also helped me understand that deploying a project is another part of software development that requires attention, just like implementing the features themselves.

Overall, I feel that this project helped me connect what we learn in class with the process of building a real application. I learned that developing software involves more than solving the main problem; it also requires making decisions about the interface, organizing the code, connecting different technologies, and making sure everything works together. I also learned that keeping things simple can be a good decision when it helps make the project easier to understand and maintain. For me, the most satisfying part was seeing how the Dijkstra algorithm could become part of a complete application instead of remaining just a concept in a programming assignment.

Valeria Pérez Maciel

For me, the most interesting part of this project was the algorithm in `dijkstra.py`. We already knew how standard Dijkstra works from class, but implementing the bidirectional version made us understand it much more deeply. It is not enough to run the same search twice: both searches have to share information to know when they meet, and the program has to know exactly when it is safe to stop. Working out why the condition $top_f + top_b \geq mejor$ guarantees that no better path can still appear was one of the moments where the theory really made sense to me.

The fact that the graph is directed also made us think about details that are easy to overlook. The backward search cannot use the same edges as the forward one, because it has to walk "against the arrow", so we decided that every `Nodo` should store both its outgoing edges (`aristas`) and its incoming edges (`entrantes`). Rebuilding the final path in two halves, one with `padre_f` and one with `padre_b`, also required care, since a small mistake there would return a path with a repeated or missing node. We also had to solve practical problems, such as the heap trying to compare two `Nodo` objects when their distances were equal, which we fixed by adding a counter as a tie-breaker. Small decisions like this, or rejecting negative weights in `Arista`, showed us that a correct algorithm also depends on handling these cases properly.

Another decision that helped us was keeping `dijkstra.py` as an independent module and testing it on its own before connecting it to the interface. We ran it with a 20×20 grid of 400 nodes and compared the result with a standard Dijkstra, which gave us confidence that the bidirectional

version was correct. Once it worked, connecting it to `app.py` was much easier, because the backend only had to rebuild the graph with our classes and call `resolver()`.

What I take away from this project is a much clearer understanding of how a pathfinding algorithm works internally, and why bidirectional Dijkstra is useful in areas like video games, where finding routes quickly matters: instead of one large search that grows in every direction, two smaller searches meet in the middle and explore far fewer nodes. I also learned the importance of testing each part separately and paying attention to edge cases, because that is what turns an idea from class into a program that actually works. I am satisfied with the final result, an application where anyone can build their own graph and see the shortest path highlighted.